

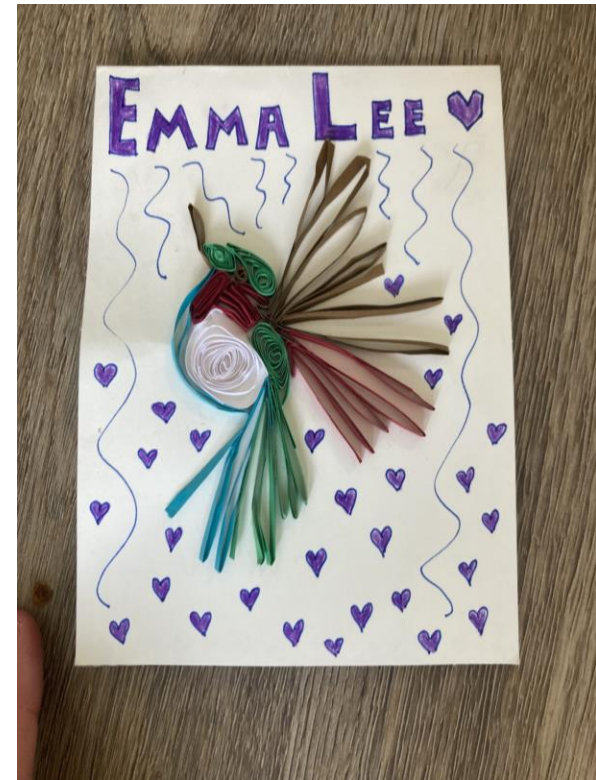


Color Palette Database

KATIE OLSEN

Overview

- Artists and Crafters need a way to store, organize, and search color palettes
- My project allows users to create, manage, and visualize color palettes stored in a MySQL database
- Built using SQL, Flask, Python, HTML, and CSS



A large, irregular pink brushstroke shape that serves as a background for the title text.

Project Summary

- A web app where users can create, view, edit, and delete custom color palettes.
- Uses a relational SQL schema with many-to-many relationships for palettes, colors, and tags.
- Frontend + backend integration through Flask, HTML/CSS/JS.

Why This Project is Interesting



Combines my love for design, art, and color with backend development.



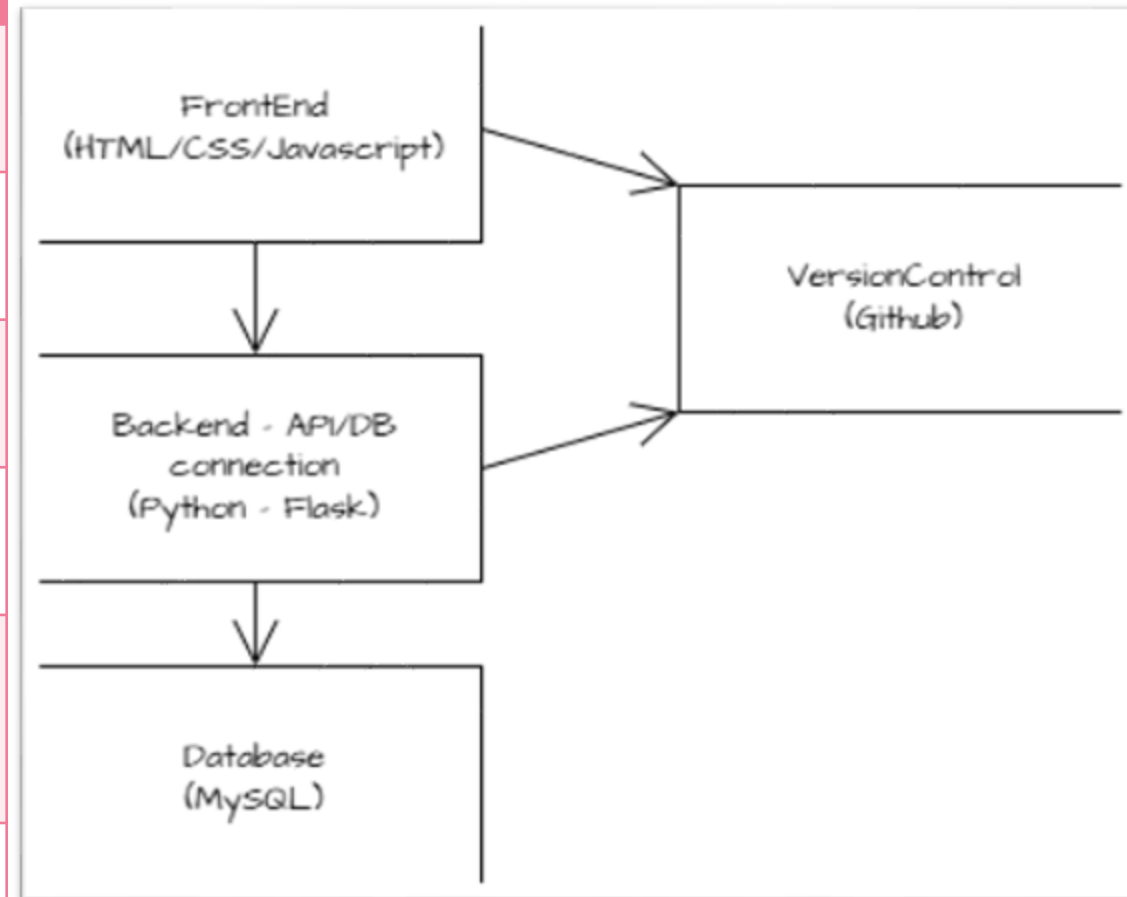
Lets me practice database design through something visual and fun.



I wanted to build a tool I'd personally use for design, notes, and inspiration.

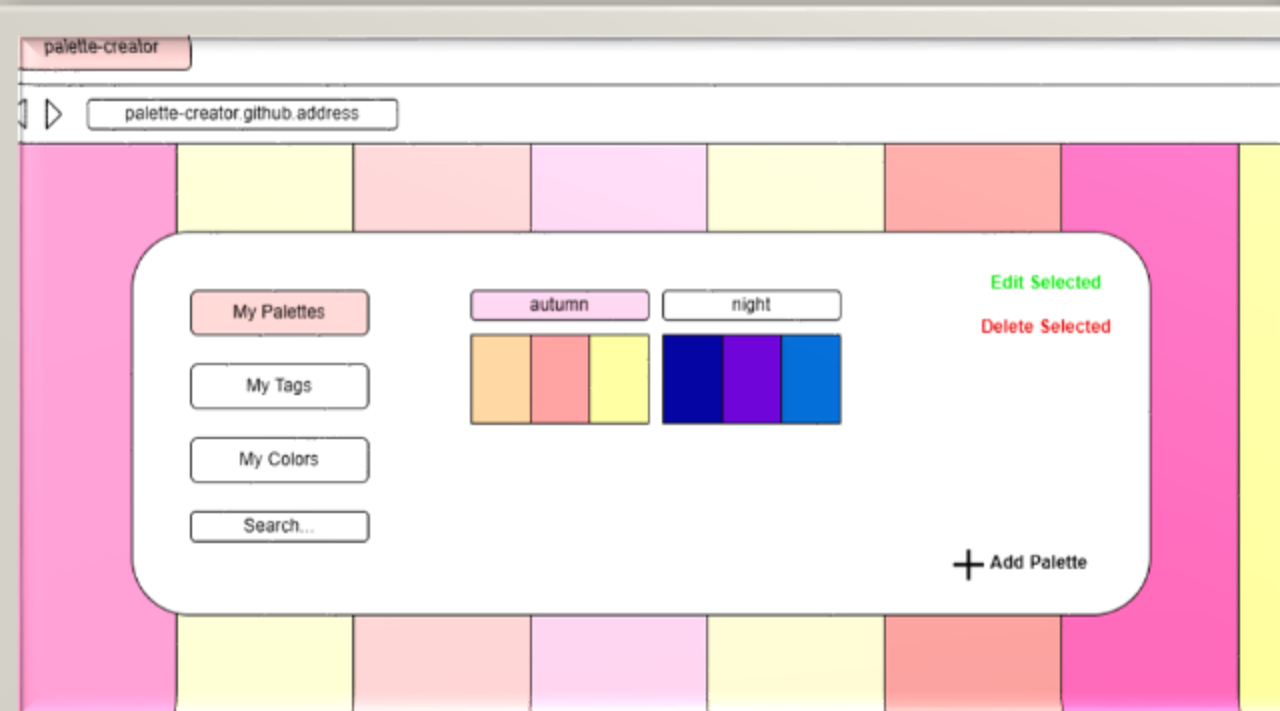
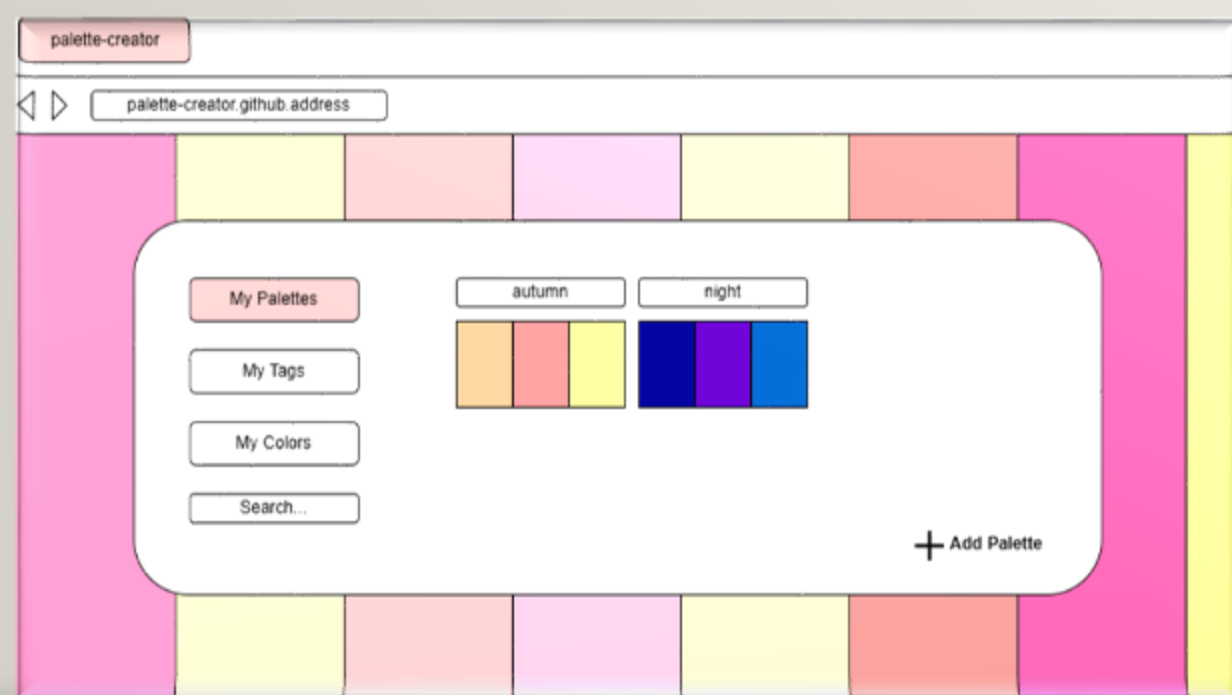
System Overview Diagram

Layer	Technology	Purpose
Database	MySql	Main SQL database
Backend	Python (Flask)	API + database connection
Frontend	HTML, CSS, JavaScript	Display palettes and forms
Version Control	Git + GitHub	Version tracking and public repo
Future - WebHosting Platform	Render or PythonAnywhere	Hosting the webpage and Database



ERD





Browser tabs: olsen, Final, Final, Color, Katie, P x, Gemini

Address bar: 127.0.0.1:5000/palettes

Navigation bar: DB, Network, CS340, Capstone, icons, All Bookmarks

Palettes

Tags

Colors

Logout

Terminal window: MINGW64/c/Users/olsen/cs452/color-palette-database

```
olsen@KatieOlsen-Lap MINGW64 ~/cs452/color-palette-database (main)
$ python app.py
* Serving Flask app 'app'
* Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment.
* Running on http://127.0.0.1:5000
Press CTRL+C to quit
* Restarting with stat
* Debugger is active!
* Debugger PIN: 364-988-653
127.0.0.1 - - [09/Dec/2025 22:45:45] "POST /login HTTP/1.1" 302 -
127.0.0.1 - - [09/Dec/2025 22:45:45] "GET /main-page HTTP/1.1" 200 -
127.0.0.1 - - [09/Dec/2025 22:45:46] "GET /static/css/base.css HTTP/1.1" 304 -
127.0.0.1 - - [09/Dec/2025 22:45:46] "GET /static/css/buttons.css HTTP/1.1" 304 -
127.0.0.1 - - [09/Dec/2025 22:45:46] "GET /static/css/layout.css HTTP/1.1" 304 -
127.0.0.1 - - [09/Dec/2025 22:45:46] "GET /static/css/effects.css HTTP/1.1" 304 -
127.0.0.1 - - [09/Dec/2025 22:45:46] "GET /static/css/sidenav.css HTTP/1.1" 304 -
127.0.0.1 - - [09/Dec/2025 22:46:04] "POST /palettes-fetch HTTP/1.1" 302 -
127.0.0.1 - - [09/Dec/2025 22:46:04] "GET /palettes HTTP/1.1" 200 -
127.0.0.1 - - [09/Dec/2025 22:46:04] "GET /static/css/base.css HTTP/1.1" 304 -
127.0.0.1 - - [09/Dec/2025 22:46:04] "GET /static/css/layout.css HTTP/1.1" 304 -
```

VS Code tabs: app.py 2 x, log.md (Working Tree) M, palette-list.html, base.html

app.py > ...

```
305
306 #####
307 # Main Pages - POST
308 #####
309
310
311 @app.route("/")
312 def index():
313     palettes = get_all_palettes()
314     return render_template("index.html", palettes=palettes)
315
316
317 @app.route("/main-page", methods=["GET", "POST"])
318 @login_required
319 def main_page():
320     username = session["username"]
321     palettes = fetch_palettes()
322     return render_template(
323         "main-page.html",
324         palettes=palettes,
325         username=username,
326         query_type=session.get("query_type", username),
327     )
328
329
330 @app.route("/palettes-fetch", methods=["POST"])
331 @login_required
332 def palettes_fetch():
333     set_query_type()
334     return redirect(url_for("palettes") or request.referrer)
335
336
337 @app.route("/tags-fetch", methods=["GET", "POST"])
338 @login_required
339 def tags_fetch():
340     set_query_type()
341     return redirect(url_for("tags") or request.referrer)
```


Welcome to palette-creator!

To begin, enter your username and password:

Username:

Password:

Login

A large, irregular pink brushstroke shape on the left side of the slide, containing the text 'What I learned'.

What I learned

- Refresher on how to use HTML/CSS/Javascript
- How to use backend routes and states in Flask
- How to organize code
- Designing and modifying a relational schema

Scaling Characteristics and Performance

Scaling

- Need to publish to PythonAnywhere (it has a MySQL database) – should help scaling. Should be able to get around 50,000 to 10,000 palette capacity
- Unique color palette names (to be implemented) could improve scalability.
- With scaling, I could add indexes and/or caching to improve performance with scaling

Performance

- Flask handles concurrency
- Authentication: simple username system: needs to be extended
- Failover: need to remove duplication in palette names, possibly color name?
- Need to test with more users.



Future Work

- Finish implementing tags and colors.
- Importing an Image (and getting palette)
- Palette Export – with hex values
- Deployment in the Cloud (through PythonAnywhere or Render)
- Password hashing



Thank you

